Guide de déploiement

> Aucun déploiement n'a été effectué : aucune URL publique n'existe encore. Ce guide décrit comment en obtenir une.

A. Local / démonstration (Docker Compose)

```
        git clone <URL-de-votre-dépôt> inclusivenue && cd inclusivenue
        cp .env.example .env
        node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"   # → coller dans JWT_SECRET
        docker compose up --build -d
        docker compose ps            # mongo, api, web : "healthy"
        docker compose logs -f api   # suivi (les e-mails du mode log y apparaissent, non envoyés)
```

Application : http://localhost:8080 — Swagger : http://localhost:8080/api-docs. Arrêt : `docker compose down` (données conservées). Ne jamais lancer `docker compose down -v` sans vouloir effacer la base.

Mise à jour : `git pull && docker compose up --build -d`. Sauvegarde : `docker compose exec mongo mongodump --archive --gzip > sauvegarde-$(date +%F).gz`.

B. Production gratuite / étudiante (recommandé : Render + MongoDB Atlas)

1. **MongoDB Atlas** (cluster M0 gratuit) : créer un utilisateur, autoriser l'IP `0.0.0.0/0` (ou les IP de Render), copier l'URI `mongodb+srv://…/inclusivenue`.
2. **API — Render « Web Service »** depuis le dépôt GitHub :
   - Root directory `backend`, runtime Docker (le `Dockerfile` est détecté) ou Node (`npm install`, `npm start`).
   - Variables : `NODE_ENV=production`, `MONGO_URL=<URI Atlas>`, `JWT_SECRET=<48 octets hex>`, `CLIENT_URL=https://<front>.onrender.com`, `PUBLIC_API_URL=https://<api>.onrender.com`, `COOKIE_SECURE=true`, `SEED_ON_START=true` (premier déploiement seulement), `MAIL_MODE=smtp` + `SMTP_*` (ou `log`).
   - Health check path : `/health`. Disque persistant monté sur `/app/uploads` (sinon les images téléversées sont perdues au redémarrage).
3. **Frontend** : le plus simple est de déployer aussi le conteneur `frontend` (nginx) en Web Service Docker, en remplaçant dans `nginx.conf` `proxy_pass http://api:4000;` par l'URL interne/publique de l'API. Alternative « Static Site » : `npm run build`, dossier `dist`, variable `VITE_API_URL=https://<api>.onrender.com` et règle de réécriture `/* → /index.html`.
   - ⚠️ Si front et API sont sur deux domaines différents, le cookie `SameSite=Strict` n'est pas envoyé : servez l'API sous le même domaine (proxy nginx, recommandé) ou passez en cookie `SameSite=None; Secure` (à adapter dans `controllers/index.js`).
4. Vérifier : `/health`, connexion avec un compte de démo, puis **changer les mots de passe des comptes de démo** ou repasser `SEED_ON_START=false` et supprimer ces comptes.

C. VPS (Docker)

Docker + Compose sur le serveur, reverse proxy HTTPS (Caddy ou Traefik) devant le port 8080, `.env` avec `COOKIE_SECURE=true` et `CLIENT_URL=https://votre-domaine`. Les images peuvent venir de GHCR (workflow `release-images.yml`, déclenché par `git tag v1.0.0 && git push --tags`).

E-mails réels

Fournisseurs SMTP gratuits pour un projet étudiant : Brevo (300/jour), Mailjet, ou Mailtrap (bac à sable de test). Renseigner `MAIL_MODE=smtp`, `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `MAIL_FROM` (adresse validée chez le fournisseur). En cas d'échec, la notification affiche « Échec de l'envoi » et l'erreur est journalisée dans `emaillogs`.

CI/CD

- `ci.yml` à chaque push / PR : lint, tests unitaires + intégration + couverture, build, `docker compose up --wait`, E2E Playwright + axe, Lighthouse CI ; artefacts téléchargeables dans l'onglet Actions.
- `release-images.yml` sur tag `v*` : images `ghcr.io/<owner>/inclusivenue-backend|frontend`, puis appel facultatif d'un *deploy hook* Render (secret `RENDER_DEPLOY_HOOK`).

Checklist sécurité avant mise en ligne

- ☐ `JWT_SECRET` aléatoire ≥ 32 caractères (l'API refuse de démarrer sinon en production)
- ☐ `COOKIE_SECURE=true` et HTTPS
- ☐ `CLIENT_URL` = origine exacte du site (CORS)
- ☐ Comptes de démo supprimés ou mots de passe changés
- ☐ Sauvegardes MongoDB planifiées
- ☐ `.env` jamais commité (déjà dans `.gitignore`)